

DecodeLabs — Data Analytics Internship

Project 3: SQL Data Analysis

Querying an E-Commerce Orders Dataset for Business Insights

Dataset: Dataset_for_Data_Analytics.xlsx | 1,200 orders | 14 columns
Tool used: SQL (SQLite dialect)

1. Objective

This project applies core SQL fundamentals — SELECT, WHERE, ORDER BY, GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG) — to an e-commerce orders dataset in order to extract actionable business intelligence. Each query below is framed as a real business question, followed by the SQL used to answer it, the resulting output, and a short insight.

2. Dataset Overview

Table name used in this analysis: Orders

Columns: OrderID, OrderDate, CustomerID, Product, Quantity, UnitPrice, ShippingAddress, PaymentMethod, OrderStatus, TrackingNumber, ItemsInCart, CouponCode, ReferralSource, TotalPrice.

The dataset contains 1,200 order records spanning 1,189 unique customers, 7 product categories, 5 order statuses, 5 payment methods, and 5 referral sources, placed between January 2023 and mid-2025.

3. A Note on SQL Execution Order

SQL is written in the order SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY, but the database engine actually evaluates it as FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. This is why column aliases created in SELECT cannot be referenced inside WHERE (WHERE runs before SELECT), but they can be used in ORDER BY (which runs last). This logic is applied consistently in the queries below — for example, HAVING is used (not WHERE) whenever a filter is applied to an aggregated value.

4. Queries, Results & Insights

1. Total number of orders and revenue by Product

SQL Query:

```
SELECT Product,
       COUNT(*) AS TotalOrders,
       SUM(TotalPrice) AS TotalRevenue,
       ROUND(AVG(TotalPrice),2) AS AvgOrderValue
FROM Orders
GROUP BY Product
ORDER BY TotalRevenue DESC;
```

Result (showing 7 of 7 rows):

Product	TotalOrders	TotalRevenue	AvgOrderValue
Chair	178	195620.11	1098.99
Printer	181	195612.61000000002	1080.73
Laptop	173	192126.56	1110.56
Tablet	179	186568.95	1042.28
Monitor	163	175651.41	1077.62
Desk	170	167459.93	985.06
Phone	156	151722.39	972.58

Insight: Revenue is fairly evenly spread across product categories (each ~13–15% of total revenue), with Chair and Printer slightly ahead. This suggests no single product dominates — a diversified catalogue.

2. Order count and revenue by OrderStatus

SQL Query:

```
SELECT OrderStatus,
       COUNT(*) AS NumOrders,
       SUM(TotalPrice) AS Revenue,
       ROUND(AVG(TotalPrice),2) AS AvgValue
FROM Orders
GROUP BY OrderStatus
ORDER BY NumOrders DESC;
```

Result (showing 5 of 5 rows):

OrderStatus	NumOrders	Revenue	AvgValue
Cancelled	250	276396.21	1105.58
Returned	247	243277.7	984.93
Pending	237	256328.15	1081.55
Shipped	235	246159.58	1047.49
Delivered	231	242600.32	1050.22

Insight: Cancelled and Returned orders together account for roughly 41% of all orders (497 of 1,200) and over Rs/\$519K in revenue at risk — a strong signal to investigate fulfilment or product-quality issues.

3. Revenue and order count by PaymentMethod

SQL Query:

```
SELECT PaymentMethod,
       COUNT(*) AS NumOrders,
       SUM(TotalPrice) AS Revenue
FROM Orders
GROUP BY PaymentMethod
ORDER BY Revenue DESC;
```

Result (showing 5 of 5 rows):

PaymentMethod	NumOrders	Revenue
Credit Card	234	263847.63
Online	258	262442.94
Cash	246	259786.29
Gift Card	230	246323.92
Debit Card	232	232361.18

Insight: Credit Card is the top revenue-generating payment method, but all five methods are within a narrow band (~Rs/\$232K–264K), showing no heavy reliance on a single payment channel.

4. All Cancelled orders paid via Credit Card, sorted by highest value

SQL Query:

```
SELECT OrderID, Product, Quantity, TotalPrice, PaymentMethod, OrderStatus
```

```
FROM Orders
WHERE OrderStatus = 'Cancelled' AND PaymentMethod = 'Credit Card'
ORDER BY TotalPrice DESC;
```

Result (showing 20 of 54 rows):

OrderID	Product	Quantity	TotalPrice	PaymentMethod	OrderStatus
ORD200527	Chair	5	3267.35	Credit Card	Cancelled
ORD200889	Monitor	5	3253.6	Credit Card	Cancelled
ORD200764	Laptop	5	3137.15	Credit Card	Cancelled
ORD200002	Tablet	5	2753.4	Credit Card	Cancelled
ORD201179	Laptop	5	2592.75	Credit Card	Cancelled
ORD200688	Monitor	4	2372.88	Credit Card	Cancelled
ORD200266	Chair	4	2332.28	Credit Card	Cancelled
ORD200089	Printer	5	2276.65	Credit Card	Cancelled
ORD200083	Laptop	4	2161.64	Credit Card	Cancelled
ORD201010	Monitor	3	1978.71	Credit Card	Cancelled
ORD200758	Tablet	3	1874.52	Credit Card	Cancelled
ORD200964	Phone	5	1822.1	Credit Card	Cancelled
ORD200564	Desk	5	1821.4	Credit Card	Cancelled
ORD200616	Tablet	3	1804.8	Credit Card	Cancelled
ORD200818	Phone	4	1563.36	Credit Card	Cancelled
ORD200228	Chair	4	1543.68	Credit Card	Cancelled
ORD200682	Monitor	3	1528.26	Credit Card	Cancelled
ORD200472	Chair	3	1524.36	Credit Card	Cancelled
ORD200966	Monitor	5	1402.6	Credit Card	Cancelled
ORD200908	Desk	2	1395.86	Credit Card	Cancelled

Insight: Filtering to Cancelled + Credit Card orders isolates 54 high-friction transactions; several exceed \$3,000 in value, making them good candidates for a closer fraud/quality audit.

5. High value orders (TotalPrice > 2000) placed in 2024

SQL Query:

```
SELECT OrderID, OrderDate, Product, TotalPrice
FROM Orders
WHERE TotalPrice > 2000 AND OrderDate LIKE '2024%'
ORDER BY TotalPrice DESC;
```

Result (showing 20 of 75 rows):

OrderID	OrderDate	Product	TotalPrice
ORD200326	2024-07-01 00:00:00	Laptop	3352.4
ORD200361	2024-06-29 00:00:00	Printer	3299.25
ORD200367	2024-04-25 00:00:00	Laptop	3293.85
ORD200527	2024-08-19 00:00:00	Chair	3267.35
ORD200889	2024-10-27 00:00:00	Monitor	3253.6
ORD200540	2024-01-29 00:00:00	Laptop	3243.25
ORD200802	2024-02-17 00:00:00	Chair	3223.2
ORD200086	2024-06-19 00:00:00	Printer	3215.15
ORD200221	2024-02-23 00:00:00	Tablet	3196.85
ORD200296	2024-06-19 00:00:00	Desk	3194.0
ORD201079	2024-06-20 00:00:00	Phone	3170.0

OrderID	OrderDate	Product	TotalPrice
ORD200764	2024-03-10 00:00:00	Laptop	3137.15
ORD200450	2024-01-15 00:00:00	Monitor	3075.5
ORD200876	2024-06-09 00:00:00	Chair	3044.55
ORD200633	2024-04-10 00:00:00	Laptop	3008.6
ORD200731	2024-06-07 00:00:00	Printer	2922.3
ORD200916	2024-06-21 00:00:00	Printer	2894.15
ORD200511	2024-03-15 00:00:00	Monitor	2876.2
ORD200816	2024-02-21 00:00:00	Tablet	2873.95
ORD200578	2024-10-10 00:00:00	Monitor	2830.35

Insight: 75 orders in 2024 exceeded \$2,000, led by Laptops and Printers — these are the premium-basket orders worth targeting with loyalty or upsell campaigns.

6. Orders shipped to addresses starting with '9' on Main St (LIKE pattern matching)

SQL Query:

```
SELECT OrderID, ShippingAddress, ItemsInCart, OrderStatus
FROM Orders
WHERE ShippingAddress LIKE '9%Main St'
ORDER BY ItemsInCart DESC
LIMIT 15;
```

Result (showing 15 of 15 rows):

OrderID	ShippingAddress	ItemsInCart	OrderStatus
ORD200180	970 Main St	10	Returned
ORD200356	907 Main St	10	Cancelled
ORD200527	967 Main St	10	Cancelled
ORD200741	945 Main St	10	Delivered
ORD200832	903 Main St	10	Pending
ORD200082	971 Main St	9	Shipped
ORD200097	922 Main St	9	Pending
ORD200217	908 Main St	9	Returned
ORD200277	910 Main St	9	Delivered
ORD200577	967 Main St	9	Returned
ORD200696	965 Main St	9	Pending
ORD200706	969 Main St	9	Returned
ORD200737	972 Main St	9	Pending
ORD200794	988 Main St	9	Delivered
ORD200798	968 Main St	9	Returned

Insight: The LIKE pattern 'starts with 9, ends with Main St' demonstrates simple substring filtering — useful for auditing a specific street/zip range for logistics planning.

7. Revenue by ReferralSource, only sources driving > 50000 in total revenue

SQL Query:

```
SELECT ReferralSource,
       COUNT(*) AS NumOrders,
       SUM(TotalPrice) AS Revenue
FROM Orders
```

```
GROUP BY ReferralSource
HAVING SUM(TotalPrice) > 50000
ORDER BY Revenue DESC;
```

Result (showing 5 of 5 rows):

ReferralSource	NumOrders	Revenue
Instagram	259	275285.45
Email	250	261808.55
Google	241	250441.48
Facebook	228	250410.9
Referral	222	226815.58

Insight: All five referral sources individually generate over \$50,000 in revenue; Instagram and Email lead, indicating marketing spend across channels is paying off fairly evenly.

8. Products with average order value greater than 1050 (HAVING on aggregate)

SQL Query:

```
SELECT Product,
       COUNT(*) AS NumOrders,
       ROUND(AVG(TotalPrice),2) AS AvgOrderValue
FROM Orders
GROUP BY Product
HAVING AVG(TotalPrice) > 1050
ORDER BY AvgOrderValue DESC;
```

Result (showing 4 of 4 rows):

Product	NumOrders	AvgOrderValue
Laptop	173	1110.56
Chair	178	1098.99
Printer	181	1080.73
Monitor	163	1077.62

Insight: Laptop, Chair, Printer, and Monitor all carry an average order value above \$1,050 — these are the products worth prioritizing in premium bundle promotions.

9. Top 10 customers by total spend

SQL Query:

```
SELECT CustomerID,
       COUNT(*) AS NumOrders,
       SUM(TotalPrice) AS TotalSpend
FROM Orders
GROUP BY CustomerID
ORDER BY TotalSpend DESC
LIMIT 10;
```

Result (showing 10 of 10 rows):

CustomerID	NumOrders	TotalSpend
C38840	2	5723.23
C57276	1	3456.4
C67260	1	3390.8
C13877	1	3384.9
C18404	1	3370.2
C16775	1	3353.75
C65986	1	3352.4
C47778	1	3334.0
C59183	1	3322.55
C25276	1	3313.9

Insight: The top 10 customers each contributed between roughly \$3,000–\$5,700 in lifetime spend — a useful shortlist for a VIP/loyalty outreach program.

10. Coupon usage - orders and revenue with vs without coupon

SQL Query:

```
SELECT COALESCE(CouponCode,'No Coupon') AS Coupon,
       COUNT(*) AS NumOrders,
       SUM(TotalPrice) AS Revenue,
       ROUND(AVG(TotalPrice),2) AS AvgOrderValue
FROM Orders
GROUP BY Coupon
ORDER BY Revenue DESC;
```

Result (showing 4 of 4 rows):

Coupon	NumOrders	Revenue	AvgOrderValue
FREESHIP	313	335036.99	1070.41
No Coupon	309	322401.41	1043.37
SAVE10	286	304840.02	1065.87
WINTER15	292	302483.54	1035.9

Insight: Orders using the FREESHIP coupon generated the highest total revenue and the highest average order value, suggesting free-shipping incentives correlate with larger baskets rather than just discount-seeking behavior.

11. Monthly order volume and revenue trend

SQL Query:

```
SELECT SUBSTR(OrderDate,1,7) AS OrderMonth,
       COUNT(*) AS NumOrders,
       SUM(TotalPrice) AS Revenue
FROM Orders
GROUP BY OrderMonth
ORDER BY OrderMonth;
```

Result (showing 20 of 30 rows):

OrderMonth	NumOrders	Revenue
2023-01	47	56685.75
2023-02	37	40117.659999999996
2023-03	43	48609.37
2023-04	31	27751.71
2023-05	49	63836.840000000004
2023-06	45	49500.19
2023-07	44	42820.659999999996
2023-08	51	54352.14
2023-09	29	29526.670000000002
2023-10	47	52607.85
2023-11	41	43079.67
2023-12	46	43754.73
2024-01	32	38528.08
2024-02	32	36909.57
2024-03	36	36030.9
2024-04	50	49613.14
2024-05	34	27909.11
2024-06	53	68068.54000000001
2024-07	43	42963.98
2024-08	28	31991.07

Insight: Monthly order volume and revenue is used to track seasonality; this series can be charted to spot peak months for staffing and inventory planning.

12. Returned or Cancelled orders count and lost revenue by Product

SQL Query:

```
SELECT Product,
       COUNT(*) AS ReturnedOrCancelled,
       SUM(TotalPrice) AS LostRevenue
FROM Orders
WHERE OrderStatus IN ('Returned','Cancelled')
GROUP BY Product
ORDER BY LostRevenue DESC;
```

Result (showing 7 of 7 rows):

Product	ReturnedOrCancelled	LostRevenue
Laptop	74	83416.02
Tablet	77	83155.82
Monitor	71	77582.04
Chair	73	75299.14
Printer	73	74614.25
Desk	67	68419.18
Phone	62	57187.46

Insight: Laptop and Tablet have the highest lost revenue from returns/cancellations (~\$83K each) — the two categories most worth investigating for quality or expectation-mismatch issues.

13. Percentage revenue contribution of each Product to total revenue

SQL Query:


```
SELECT Product,
       SUM(TotalPrice) AS Revenue,
       ROUND(100.0 * SUM(TotalPrice) / (SELECT SUM(TotalPrice) FROM Orders), 2) AS
PctOfTotalRevenue
FROM Orders
GROUP BY Product
ORDER BY PctOfTotalRevenue DESC;
```

Result (showing 7 of 7 rows):

Product	Revenue	PctOfTotalRevenue
Printer	195612.61000000002	15.47
Chair	195620.11	15.47
Laptop	192126.56	15.19
Tablet	186568.95	14.75
Monitor	175651.41	13.89
Desk	167459.93	13.24
Phone	151722.39	12.0

Insight: No single product exceeds ~15.5% of total revenue, confirming a balanced product mix rather than dependency on one bestseller.

14. Overall summary aggregates (single-row KPI query)

SQL Query:

```
SELECT COUNT(*) AS TotalOrders,
       COUNT(DISTINCT CustomerID) AS UniqueCustomers,
       SUM(TotalPrice) AS TotalRevenue,
       ROUND(AVG(TotalPrice),2) AS AvgOrderValue,
       SUM(Quantity) AS TotalUnitsSold
FROM Orders;
```

Result (showing 1 of 1 row):

TotalOrders	UniqueCustomers	TotalRevenue	AvgOrderValue	TotalUnitsSold
1200	1189	1264761.96	1053.97	3535

Insight: At a glance: 1,200 orders, 1,189 unique customers, \$1.26M in total revenue, an average order value of \$1,053.97, and 3,535 total units sold.

5. Conclusion

Across 14 queries, this analysis moved from row-level filtering (WHERE, LIKE) to categorical bucketing (GROUP BY) to aggregated business metrics (COUNT, SUM, AVG) and finally to derived KPIs such as percentage revenue contribution — following the database's real FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY execution path rather than the order the SQL happens to be typed in.

Key takeaways for the business: (1) Cancelled/Returned orders represent ~41% of volume and are the single biggest revenue-recovery opportunity; (2) revenue is well diversified across products, payment methods, and referral channels, reducing single-point business risk; (3) a small set of top customers and premium product categories are the clearest targets for loyalty and upsell programs.

This completes the SQL fundamentals milestone (SELECT, WHERE, ORDER BY, GROUP BY, HAVING, COUNT/SUM/AVG) required for Project 3 of the DecodeLabs Data Analytics Industrial Training Kit.